



Reduction (5.1 - 5.3)

COT 4210 Discrete Structures II
Summer 2025
Department of Computer Science
Dr. Steinberg

Reduction

- A **reduction** is a way to turn one problem into another problem. A bit more specifically, it is a way to:
 - Turn problem A into problem B
 - Given the solution to problem B, solve problem A
- For example, with a basic knowledge of geometry, we can **reduce** the problem of measuring a rectangle to the problem of measuring its length and width.
- Reduction is important to decidability and recognizability because of the following observation: In general, **if**:
 - If we know we **cannot** decide A, **and**
 - We can **reduce** deciding A to deciding B, **then**
 - **We know we cannot decide B.**
- Let's use one of the most iconic observations in computer science as an example.

The Halting Problem for Turing Machines

Let $HALT_{TM} = \{ \langle M, s \rangle \mid M \text{ is a Turing Machine that halts on input } s \}$.

The observation we are going to make is that, if this problem is decidable, so is A_{TM} .

Remember that A_{TM} is easily *recognizable*. We can always correctly accept; we can't always correctly reject. What we can observe – almost trivially – is that a working $HALT_{TM}$ **allows us to correctly reject** for A_{TM} .

We assume by way of contradiction that $HALT_{TM}$ is decidable, and build our machine to decide A_{TM} .

Assume BWOC that TM M_{HALT} decides $HALT_{TM}$. Construct the following machine M_A to decide A_{TM} for machine M and input s :

M_A : On input $\langle M, s \rangle$:

- Run M_{HALT} on $\langle M, s \rangle$ to determine whether M will halt on s .
- If M_{HALT} rejects, then M will not halt on s , and hence s is not in $L(M)$. Reject.
- If M_{HALT} accepts, then we can simulate M on s until it halts. Do so.
- Reject or accept as does m .

This machine easily decides A_{TM} ; but A_{TM} is proven undecidable. $\rightarrow \leftarrow$

Therefore, **$HALT_{TM}$ is undecidable.**

Emptiness of Turing Machines

- Let $E_{\text{TM}} = \{ \langle M, s \rangle \mid M \text{ is a Turing Machine and } L(M) = \Phi \}$.
- It turns out this is undecidable too, but we can't determine that the same way as the halting proof; walk through it, and we would just determine that M accepts *something*.
- What we're going to use instead is a trick that we're going to call an **inline Turing construction** proof.
 - This kind of proof focuses on the idea that *a Turing machine can build another Turing machine*.
 - (Yes, it can. Compilers exist.)
 - Once we've built this Turing machine, we then exploit its properties to prove that E_{TM} is undecidable.
- This is a case where the idea is clearer embodied in an example, so let's get to it.

Emptiness of Turing Machines

Assume BWOC that TM M_E decides E_{TM} .
Construct the following machine M_A to decide A_{TM} for machine M and input s :

M_A : On input $\langle M, s \rangle$:

- Build a new machine M_1 :

M_1 : On input x :

- Reject if $x \neq s$.
 - Otherwise, simulate M on s , and accept or reject as it does.
- Run M_E on M_1 .
 - Accept if M_E rejects; reject if M_E accepts.

What we do here is, *within our Turing machine*, build a *new Turing machine*.

This machine, M_1 , is:

- Just like the machine M we are testing for emptiness, except:
- It **only** potentially accepts s – the original input to our Turing machine. Which means that...
- **Either it accepts s , or its language is empty.**
- We then use M_E to interrogate it decisively.
- If we can do that for our input string, we can do it for any string.
- We've solved the acceptance problem, and thus arrived at contradiction. Let's clean up.

Emptiness of Turing Machines

Assume BWOC that TM M_E decides E_{TM} .
Construct the following machine M_A to decide A_{TM} for machine M and input s :

M_A : On input $\langle M, s \rangle$:

- Build a new machine M_1 :

M_1 : On input x :

- Reject if $x \neq s$.
- Otherwise, simulate M on s , and accept or reject as it does.

- Run M_E on M_1 .
- Accept if M_E rejects; reject if M_E accepts.

M_A constructs a machine M_1 that either:

- Accepts only the original input string s , if and only if input machine M accepts it; or
- Accepts nothing.

The language of M_1 is therefore nonempty if and only if M accepts s .

M_A interrogates it using M_E , determining decisively whether M accepts s ; it therefore decides A_{TM} . $\rightarrow \leftarrow$

We conclude that E_{TM} is undecidable.

Emptiness of Turing Machines: More on Inline Turing Construction

Assume BWOC that TM M_E decides E_{TM} .
Construct the following machine M_A to decide A_{TM} for machine M and input s :

M_A : On input $\langle M, s \rangle$:

- Build a new machine M_1 :

M_1 : On input x :

- Reject if $x \neq s$.
 - Otherwise, simulate M on s , and accept or reject as it does.
- Run M_E on M_1 .
 - Accept if M_E rejects; reject if M_E accepts.

Make sure you properly keep track of all the players in this program; they'll appear in all inline Turing construction proofs.

- E_{TM} is the language we are trying to prove is undecidable in the first place.
- M_E is the machine that we assume exists to decide it, by way of contradiction.
- M_A is the machine that we create to decide A_{TM} , proving the contradiction.
- A_{TM} is the general acceptance problem for Turing machines, which we know is undecidable.
- M_1 is the machine that M_A creates in the process of its execution, that exploits the properties of E_{TM} to make A_{TM} decidable by way of M_E .

Regularity of Turing Machines

Let $REGULAR_{TM} = \{ \langle M, s \rangle \mid M \text{ is a Turing Machine and } L(M) \text{ is regular} \}$.

This proof works just like the one for emptiness does, except M_1 is exploiting regularity rather than emptiness. It's confusing until you realize the following:

- M_1 accepts **everything** if M accepts s . Its language becomes Σ^* , which is regular.
- M_1 accepts $0^n 1^n$ if M doesn't. Its language becomes the iconic irregular language.

With this realization, the explanation follows easily and the proof is complete.

Assume BWOC that TM M_{REG} decides $REGULAR_{TM}$. Construct the following machine M_A to decide A_{TM} for machine M and input s :

M_A : On input $\langle M, s \rangle$:

- Build a new machine M_1 :

M_1 : On input x :

- Accept if x has the form $0^n 1^n$.
- Otherwise, simulate M on s , and accept or reject as it does.

- Run M_{REG} on M_1 .
- Accept if M_{REG} accepts; reject if M_{REG} rejects.

Equivalence of Turing Machines

Let $EQ_{TM} = \{ \langle M_X, M_Y \rangle \mid M_X \text{ and } M_Y \text{ are Turing machines with } L(M_X) = L(M_Y) \}$.

This one's an inline Turing construction proof where we reduce from something other than the acceptance problem – in a way that completely trivializes the issue.

If we can tell if two Turing machines are equivalent, it's *real* easy to tell if one's empty.

Remember that when you're trying to show undecidability, you can reduce from *any* problem that we already know is undecidable.

Assume BWOC that TM M_{EQ} decides E_{TM} . Construct the following machine M_E to decide E_{TM} for machine M and input s :

M_E : On input $\langle M, s \rangle$:

- Build a new machine M_1 :

M_1 : On input x :

- Reject.

- Run M_{EQ} on $\langle M, M_1 \rangle$.
- Accept if M_{EQ} accepts; reject if M_{EQ} rejects.

Computation Histories

- A **computation history** is a sequence of configurations of a Turing machine M on a given input s :

$$C_1, C_2, C_3, \dots, C_F$$

- So that:
 - C_1 is the start configuration of M on s .
 - C_i transitions legally to C_{i+1} for all relevant i .
 - C_F is a halting configuration (i.e., its state is either q_{ACCEPT} or q_{REJECT}).
- **Accepting** and **rejecting** computation histories are exactly what you think they are.
- Computation histories are **finite** sequences. That means that if M doesn't halt on s , there is no computation history for M on s .

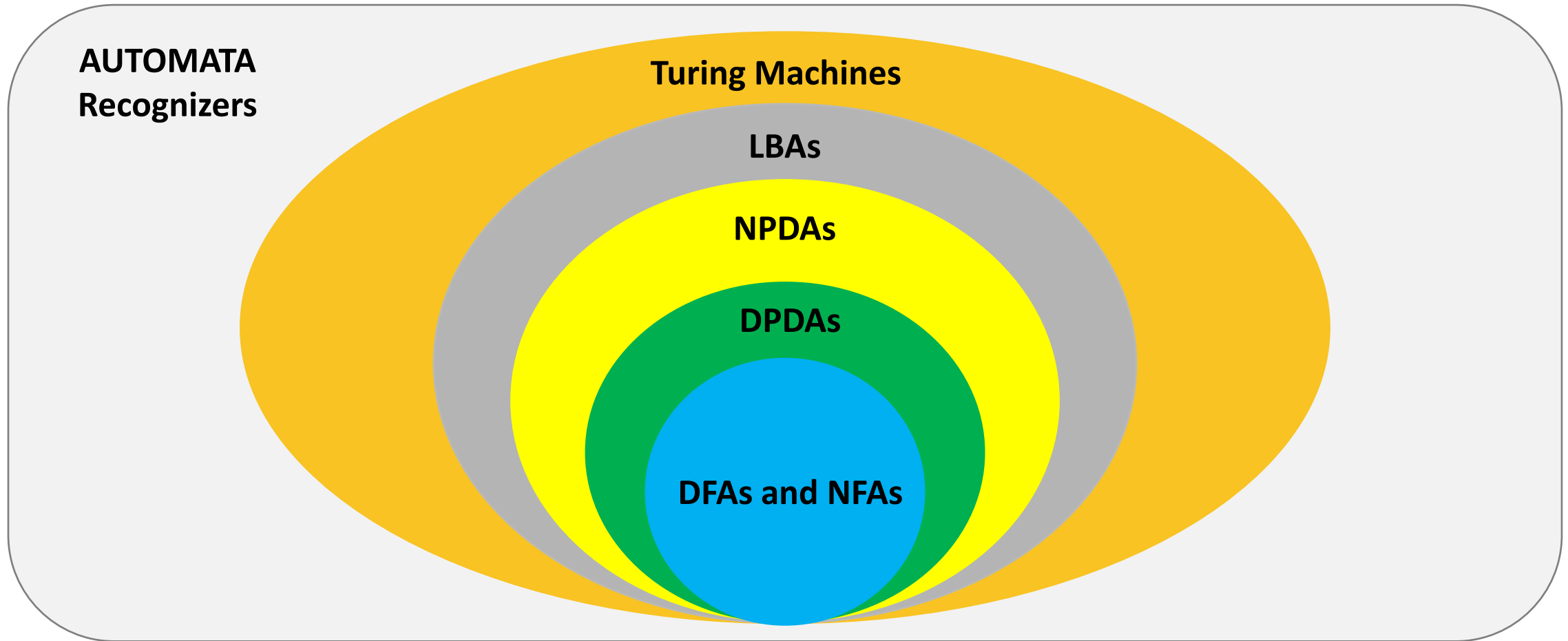
Linear-Bounded Automata

- A **linear bounded automaton** is a TM that can't move its head off the input to the right.
 - Given an LBA with q states, g symbols in the tape alphabet and an input of length n , then there are a maximum of qng^n configurations of that LBA.
 - This means that A_{LBA} **is** a decidable language:
 - Run the LBA for qng^n steps.
 - If it accepts, accept. If it rejects, reject.
 - If it hasn't either accepted or rejected, then by the pigeonhole principle, it's repeated a configuration. We can observe with a bit of thought that **a Turing machine that repeats a configuration can never halt.**
 - So if the LBA hasn't accepted after qng^n steps, reject.
- LBAs are good for several things. In particular, any CFL – and hence any regular language – can be decided by an LBA.
 - They also are (theoretically – very theoretically) exhaustively decidable.
 - That doesn't mean they get out of all the other problems, though...

Undecidability of E_{LBA}

- As usual, we prove that E_{LBA} is undecidable by a reduction from A_{TM} . The trick this time is that we do it using a **computation history** of a Turing machine M with its input s .
- Consider the construction of an LBA $B(M, s)$ that accepts **accepting computation histories** for a particular Turing machine M . That LBA does the following:
 - Takes $\langle C_1, C_2, C_3, \dots, C_F \rangle$ as input.
 - Verifies that C_1 is the start configuration of M on s .
 - Verifies that each C_{i+1} follows from its preceding C_i .
 - Verifies that C_F is an accepting configuration for M .
- If we know M and s , then $B(M, s)$ can be created algorithmically. But if $B(M, s)$ exists, it is nonempty if and only if there is an accepting computation history for M on s – in other words, if M accepts s . The reduction of A_{TM} follows from there.

MODELS OF COMPUTATION



The Post Correspondence Problem

- Now we turn to a more interesting problem of computation histories. **Post's Correspondence Problem**, or the **Post Correspondence Problem**, is most easily described in terms of dominoes.
- We have a set of dominoes. Each domino has a string on the top and the bottom, and looks something like this: $\left[\frac{abc}{c} \right]$
- So the set looks like this: $\left\{ \left[\frac{b}{ca} \right], \left[\frac{a}{ab} \right], \left[\frac{ca}{a} \right], \left[\frac{abc}{c} \right] \right\}$
- The idea is to find a **match**: make a list of the dominoes so that the string across the top is the same as the one across bottom. For example, our list has this match: $\left[\frac{a}{ab} \right] \left[\frac{b}{ca} \right] \left[\frac{ca}{a} \right] \left[\frac{a}{ab} \right] \left[\frac{abc}{c} \right]$
- It turns out the problem of figuring out if the set has a match is completely unsolvable – at least, algorithmically.

Undecidability of PCP:

General Idea

- First, it's easy enough to express the problem formally as a language:
- An instance of the PCP is a collection of dominoes $P = \left\{ \begin{bmatrix} t_1 \\ b_1 \end{bmatrix}, \begin{bmatrix} t_2 \\ b_2 \end{bmatrix}, \dots, \begin{bmatrix} t_k \\ b_k \end{bmatrix} \right\}$. A match is a sequence $i_1 i_2 \dots i_n$ where $t_{i_1} t_{i_2} \dots t_{i_n} = b_{i_1} b_{i_2} \dots b_{i_n}$. Then:
 - $PCP = \{ \langle P \rangle \mid P \text{ is an instance of the Post Correspondence Problem with a match.} \}$
- What we're going to do is the following:
 - Take a Turing machine M and an input string w .
 - Construct an instance $P(M, w)$ of the Post Correspondence Problem so that **a match for it is an accepting computation history for M on w .**
 - Therefore, if $P(M, w)$ has a match, we know whether M has an accepting computation history on w .
 - Therefore, if we can decide PCP, we can decide A_{TM} .
 - *We can't do that.*
 - Therefore, we can't decide PCP.
- To get there, we need to create $P(M, w)$.

Undecidability of PCP: Some Restrictions

- To make this proof a lot less messy, we're going to set up two restrictions.
 - First, we assume that ***M never tries to move off the left edge of the tape.*** Modifying M so it never does this is annoying but straightforward (hint: think of a beginning-of-tape symbol).
 - Second, we create a new version of PCP such that ***a match always has to start with the first domino.*** Here's how we do this:

- Take an instance of the PCP.
- Add two unique symbols for these purposes.
We'll call them $\star$ and $\triangle$.

$$\left\{ \left[\frac{t_1}{b_1} \right], \left[\frac{t_2}{b_2} \right], \dots, \left[\frac{t_k}{b_k} \right] \right\}$$

- Add them to the dominoes like so:

$$\left\{ \left[\frac{\star t_1}{\star b_1 \star} \right], \left[\frac{\star t_1}{b_1 \star} \right], \left[\frac{\star t_2}{b_2 \star} \right], \dots, \left[\frac{\star t_k}{b_k \star} \right], \left[\frac{\star \triangle}{\triangle} \right] \right\}$$

- Since only the first domino has a star at the beginning of its bottom string, it's the only one that can start the match. The triangle domino does nothing but act as our right fencepost.
- We'll call this version MPCP.

Undecidability of PCP: Starting the Machine

- Define M in the usual way, and make the first domino like so:

Rule 1: $\left[\frac{\#}{\#q_0w_1w_2...w_n\#} \right]$ is the first domino.

- This represents the start configuration of M on w .
- We're going to represent the configurations exactly like we write them in the uqv format – the string before the head, then the state, then the string at and after the head.
- $\#$ represents the separation between configurations.

Undecidability of PCP: The Tape Head

- We need to be able to handle moving the tape head left and right. (Remember that we decided not to worry about the case of the head trying to go off the left edge.)
- Let's move right first. For every a and b in the tape alphabet, and every state q and r where q isn't q_{REJECT} :

Rule 2: If $\delta(q, a) = (r, b, R)$, $\begin{bmatrix} qa \\ br \end{bmatrix}$ is a domino.

- Now let's handle moving left. For every a , b , **and** c in the tape alphabet, and every state q and r where q isn't q_{REJECT} :

Rule 3: If $\delta(q, a) = (r, b, L)$, $\begin{bmatrix} cqa \\ rcb \end{bmatrix}$ is a domino.

- Let's also take care of *not* changing anything *e*/se on the tape. For every a in the tape alphabet,

Rule 4: $\begin{bmatrix} a \\ a \end{bmatrix}$ is a domino.

Undecidability of PCP: Wind it Up, Watch it Go

Rule 1: $\left[\frac{\#}{\#q_0w_1w_2...w_n\#} \right]$

Rule 2: If $\delta(q, a) = (r, b, R)$, $\left[\frac{qa}{br} \right]$ is a domino.

Rule 3: If $\delta(q, a) = (r, b, L)$, $\left[\frac{cqa}{rcb} \right]$ is a domino.

Rule 4: $\left[\frac{a}{a} \right]$ is a domino.

So let's say we have $\delta(q_0, 0) = (q_3, 1, R)$ and $w = 0110$. Here's what happens:

Undecidability of PCP: Wind it Up, Watch it Go

Rule 1: $\left[\frac{\#}{\#q_0w_1w_2\dots w_n\#} \right]$

Rule 2: If $\delta(q, a) = (r, b, R)$, $\left[\frac{qa}{br} \right]$ is a domino.

Rule 3: If $\delta(q, a) = (r, b, L)$, $\left[\frac{cqa}{rcb} \right]$ is a domino.

Rule 4: $\left[\frac{a}{a} \right]$ is a domino.

So let's say we have $\delta(q_0, 0) = (q_3, 1, R)$ and $w = 0110$. Here's what happens:

Starting configuration q_00110

$$\left[\frac{\#}{\#q_00110\#} \right]$$

Undecidability of PCP: Wind it Up, Watch it Go

Rule 1: $\left[\frac{\#}{\#q_0w_1w_2...w_n\#} \right]$

Rule 2: If $\delta(q, a) = (r, b, R)$, $\left[\frac{qa}{br} \right]$ is a domino.

Rule 3: If $\delta(q, a) = (r, b, L)$, $\left[\frac{cqa}{rcb} \right]$ is a domino.

Rule 4: $\left[\frac{a}{a} \right]$ is a domino.

So let's say we have $\delta(q_0, 0) = (q_3, 1, R)$ and $w = 0110$. Here's what happens:

Act on q_0 , and write 1 over 0

$$\left[\frac{\#}{\#q_00110\#} \right] \left[\frac{q_00}{1q_3} \right]$$

Undecidability of PCP: Wind it Up, Watch it Go

Rule 1: $\left[\frac{\#}{\#q_0w_1w_2...w_n\#} \right]$

Rule 2: If $\delta(q, a) = (r, b, R)$, $\left[\frac{qa}{br} \right]$ is a domino.

Rule 3: If $\delta(q, a) = (r, b, L)$, $\left[\frac{cqa}{rcb} \right]$ is a domino.

Rule 4: $\left[\frac{a}{a} \right]$ is a domino.

So let's say we have $\delta(q_0, 0) = (q_3, 1, R)$ and $w = 0110$. Here's what happens:

Match the 1

$$\left[\frac{\#}{\#q_00110\#} \right] \left[\frac{q_00}{1q_3} \right] \left[\frac{1}{1} \right]$$

Undecidability of PCP: Wind it Up, Watch it Go

Rule 1: $\left[\frac{\#}{\#q_0w_1w_2...w_n\#} \right]$

Rule 2: If $\delta(q, a) = (r, b, R)$, $\left[\frac{qa}{br} \right]$ is a domino.

Rule 3: If $\delta(q, a) = (r, b, L)$, $\left[\frac{cqa}{rcb} \right]$ is a domino.

Rule 4: $\left[\frac{a}{a} \right]$ is a domino.

So let's say we have $\delta(q_0, 0) = (q_3, 1, R)$ and $w = 0110$. Here's what happens:

Match the other 1

$$\left[\frac{\#}{\#q_00110\#} \right] \left[\frac{q_00}{1q_3} \right] \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Undecidability of PCP: Wind it Up, Watch it Go

Rule 1: $\left[\frac{\#}{\#q_0w_1w_2...w_n\#} \right]$

Rule 2: If $\delta(q, a) = (r, b, R)$, $\left[\frac{qa}{br} \right]$ is a domino.

Rule 3: If $\delta(q, a) = (r, b, L)$, $\left[\frac{cqa}{rcb} \right]$ is a domino.

Rule 4: $\left[\frac{a}{a} \right]$ is a domino.

So let's say we have $\delta(q_0, 0) = (q_3, 1, R)$ and $w = 0110$. Here's what happens:

Match the 0

$$\left[\frac{\#}{\#q_00110\#} \right] \left[\frac{q_00}{1q_3} \right] \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Undecidability of PCP: Wind it Up, Watch it Go

Rule 1: $\left[\frac{\#}{\#q_0w_1w_2...w_n\#} \right]$

Rule 2: If $\delta(q, a) = (r, b, R)$, $\left[\frac{qa}{br} \right]$ is a domino.

Rule 3: If $\delta(q, a) = (r, b, L)$, $\left[\frac{cqa}{rcb} \right]$ is a domino.

Rule 4: $\left[\frac{a}{a} \right]$ is a domino.

So let's say we have $\delta(q_0, 0) = (q_3, 1, R)$ and $w = 0110$. Here's what happens:

Match the 0

$\#q_00110$

$\left[\frac{\#}{\#q_00110\#} \right] \left[\frac{q_00}{1q_3} \right] \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

$\#q_00110\#1q_3110$

So we've represented q_00110 yielding $1q_3110$ – which is exactly what would happen.

Undecidability of PCP: Iteration of Configurations

Now let's wrap up each step of the Turing machine.

Rule 5: $\begin{bmatrix} \# \\ - \\ \# \end{bmatrix}$ and $\begin{bmatrix} \# \\ \square \\ \# \end{bmatrix}$ are dominoes.

That other thing in the second one is the blank symbol, if you're wondering.

- It lets us handle various things that can happen if we go off the tape to the right.
- Going off the tape to the left would be really annoying. That's why we don't do that.

$$\begin{bmatrix} \# \\ \#q_00110\# \end{bmatrix} \begin{bmatrix} q_00 \\ 1q_3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Undecidability of PCP: Iteration of Configurations

Now let's wrap up each step of the Turing machine.

Rule 5: $\begin{bmatrix} \# \\ - \\ \# \end{bmatrix}$ and $\begin{bmatrix} \# \\ \square \\ \# \end{bmatrix}$ are dominoes.

That other thing in the second one is the blank symbol, if you're wondering.

- It lets us handle various things that can happen if we go off the tape to the right.
- Going off the tape to the left would be really annoying. That's why we don't do that.

Finish the Configuration

$$\begin{bmatrix} \# \\ - \\ \#q_00110\# \end{bmatrix} \begin{bmatrix} q_00 \\ 1q_3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} \# \\ - \\ \# \end{bmatrix}$$

As before, we've represented q_00110 yielding $1q_3110$. Now we add the configuration delimiter.

Undecidability of PCP: Finishing Up

Looking at it this way it's surprisingly obvious how we're simulating the Turing machine. All that's left is to accept. The way we do this is by allowing the accept state to "eat" other symbols. For every a in the tape alphabet,

Rule 6: $\left[\frac{a q_{\text{ACCEPT}}}{q_{\text{ACCEPT}}} \right]$ and $\left[\frac{q_{\text{ACCEPT}} a}{q_{\text{ACCEPT}}} \right]$ are dominoes.

Undecidability of PCP: Finishing Up

Looking at it this way it's surprisingly obvious how we're simulating the Turing machine. All that's left is to accept. The way we do this is by allowing the accept state to "eat" other symbols. For every a in the tape alphabet,

Rule 6: $\left[\frac{a q_{\text{ACCEPT}}}{q_{\text{ACCEPT}}} \right]$ and $\left[\frac{q_{\text{ACCEPT}} a}{q_{\text{ACCEPT}}} \right]$ are dominoes.

Let's say q_3 is q_{ACCEPT} and watch what happens.

$\#q_00110\#$

$$\left[\frac{\#}{\#q_00110\#} \right] \left[\frac{q_00}{1q_A} \right] \left[\frac{1}{1} \right] \left[\frac{1}{1} \right] \left[\frac{0}{0} \right] \left[\frac{\#}{\#} \right]$$

$\#q_00110\#1q_A110\#$

Undecidability of PCP: Finishing Up

Looking at it this way it's surprisingly obvious how we're simulating the Turing machine. All that's left is to accept. The way we do this is by allowing the accept state to "eat" other symbols. For every a in the tape alphabet,

Rule 6: $\left[\frac{a q_{\text{ACCEPT}}}{q_{\text{ACCEPT}}} \right]$ and $\left[\frac{q_{\text{ACCEPT}} a}{q_{\text{ACCEPT}}} \right]$ are dominoes.

Let's say q_3 is q_{ACCEPT} and watch what happens.

$\#q_00110\#1q_A110$

$$\left[\frac{\#}{\#q_00110\#} \right] \left[\frac{q_00}{1q_A} \right] \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} \# \\ \# \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \left[\frac{q_A1}{q_A} \right] \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$\#q_00110\#1q_A110\#1q_A10$

Undecidability of PCP: Finishing Up

Looking at it this way it's surprisingly obvious how we're simulating the Turing machine. All that's left is to accept. The way we do this is by allowing the accept state to "eat" other symbols. For every a in the tape alphabet,

Rule 6: $\left[\frac{a q_{\text{ACCEPT}}}{q_{\text{ACCEPT}}} \right]$ and $\left[\frac{q_{\text{ACCEPT}} a}{q_{\text{ACCEPT}}} \right]$ are dominoes.

Let's say q_3 is q_{ACCEPT} and watch what happens.

$\#q_00110\#1q_A110\#1q_A10$

$$\left[\frac{\#}{\#q_00110\#} \right] \left[\frac{q_00}{1q_A} \right] \left[\frac{1}{1} \right] \left[\frac{1}{1} \right] \left[\frac{0}{0} \right] \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A1}{q_A} \right] \left[\frac{1}{1} \right] \left[\frac{0}{0} \right] \dots$$

$$\dots \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A1}{q_A} \right] \left[\frac{0}{0} \right]$$

$\#q_00110\#1q_A110\#1q_A10\#1q_A0$

Undecidability of PCP: Finishing Up

Looking at it this way it's surprisingly obvious how we're simulating the Turing machine. All that's left is to accept. The way we do this is by allowing the accept state to "eat" other symbols. For every a in the tape alphabet,

Rule 6: $\left[\frac{a q_{\text{ACCEPT}}}{q_{\text{ACCEPT}}} \right]$ and $\left[\frac{q_{\text{ACCEPT}} a}{q_{\text{ACCEPT}}} \right]$ are dominoes.

Let's say q_3 is q_{ACCEPT} and watch what happens.

$\#q_00110\#1q_A110\#1q_A10\#1q_A0$

$\left[\frac{\#}{\#q_00110\#} \right] \left[\frac{q_00}{1q_A} \right] \left[\frac{1}{1} \right] \left[\frac{1}{1} \right] \left[\frac{0}{0} \right] \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A1}{q_A} \right] \left[\frac{1}{1} \right] \left[\frac{0}{0} \right] \dots$

$\dots \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A1}{q_A} \right] \left[\frac{0}{0} \right] \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A0}{q_A} \right]$

$\#q_00110\#1q_A110\#1q_A10\#1q_A0\#1q_A$

Undecidability of PCP: Finishing Up

Looking at it this way it's surprisingly obvious how we're simulating the Turing machine. All that's left is to accept. The way we do this is by allowing the accept state to "eat" other symbols. For every a in the tape alphabet,

Rule 6: $\left[\frac{a q_{\text{ACCEPT}}}{q_{\text{ACCEPT}}} \right]$ and $\left[\frac{q_{\text{ACCEPT}} a}{q_{\text{ACCEPT}}} \right]$ are dominoes.

Let's say q_3 is q_{ACCEPT} and watch what happens.

$\#q_00110\#1q_A110\#1q_A10\#1q_A0\#1q_A$

$\left[\frac{\#}{\#q_00110\#} \right] \left[\frac{q_00}{1q_A} \right] \left[\frac{1}{1} \right] \left[\frac{1}{1} \right] \left[\frac{0}{0} \right] \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A1}{q_A} \right] \left[\frac{1}{1} \right] \left[\frac{0}{0} \right] \dots$

$\dots \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A1}{q_A} \right] \left[\frac{0}{0} \right] \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A0}{q_A} \right] \left[\frac{\#}{\#} \right] \left[\frac{1q_A}{q_A} \right]$

$\#q_00110\#1q_A110\#1q_A10\#1q_A0\#1q_A\#q_A$

Undecidability of PCP: Finishing Up

Looking at it this way it's surprisingly obvious how we're simulating the Turing machine. All that's left is to accept. The way we do this is by allowing the accept state to "eat" other symbols. For every a in the tape alphabet,

Rule 6: $\left[\frac{a q_{\text{ACCEPT}}}{q_{\text{ACCEPT}}} \right]$ and $\left[\frac{q_{\text{ACCEPT}} a}{q_{\text{ACCEPT}}} \right]$ are dominoes.

Let's say q_3 is q_{ACCEPT} and watch what happens.

....wait, now what?

$\#q_00110\#1q_A110\#1q_A10\#1q_A0\#1q_A\#$

$\left[\frac{\#}{\#q_00110\#} \right] \left[\frac{q_00}{1q_A} \right] \left[\frac{1}{1} \right] \left[\frac{1}{1} \right] \left[\frac{0}{0} \right] \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A1}{q_A} \right] \left[\frac{1}{1} \right] \left[\frac{0}{0} \right] \dots$

$\dots \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A1}{q_A} \right] \left[\frac{0}{0} \right] \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A0}{q_A} \right] \left[\frac{\#}{\#} \right] \left[\frac{1q_A}{q_A} \right] \left[\frac{\#}{\#} \right]$

$\#q_00110\#1q_A110\#1q_A10\#1q_A0\#1q_A\#q_A\#$

Undecidability of PCP: Finishing Up

Looking at it this way it's surprisingly obvious how we're simulating the Turing machine. All that's left is to accept. The way we do this is by allowing the accept state to "eat" other symbols. For every a in the tape alphabet,

Rule 6: $\left[\frac{a q_{\text{ACCEPT}}}{q_{\text{ACCEPT}}} \right]$ and $\left[\frac{q_{\text{ACCEPT}} a}{q_{\text{ACCEPT}}} \right]$ are dominoes.

Let's say q_3 is q_{ACCEPT} and watch what happens.

....wait, now what?

One last rule allows us to complete the match.

Rule 7: $\left[\frac{q_{\text{ACCEPT}}\#\#}{\#} \right]$ is a domino.

$\#q_00110\#1q_A110\#1q_A10\#1q_A0\#1q_A\#q_A\#\#$

$\left[\frac{\#}{\#q_00110\#} \right] \left[\frac{q_00}{1q_A} \right] \left[\frac{1}{1} \right] \left[\frac{1}{1} \right] \left[\frac{0}{0} \right] \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A1}{q_A} \right] \left[\frac{1}{1} \right] \left[\frac{0}{0} \right] \dots$

$\dots \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A1}{q_A} \right] \left[\frac{0}{0} \right] \left[\frac{\#}{\#} \right] \left[\frac{1}{1} \right] \left[\frac{q_A0}{q_A} \right] \left[\frac{\#}{\#} \right] \left[\frac{1q_A}{q_A} \right] \left[\frac{\#}{\#} \right] \left[\frac{q_A\#\#}{\#} \right]$

$\#q_00110\#1q_A110\#1q_A10\#1q_A0\#1q_A\#q_A\#\#$

Undecidability of PCP: Cleanup

- We now have an instance of PCP, namely $P(M, w)$, that has a match if and only if M has an accepting configuration history on w . We can now prove that PCP is undecidable; the proof is, of course, contradiction by reduction from A_{TM} .
- Assume that PCP is decidable. That means it is decided by a Turing machine, which we'll imaginatively name PCP . We can make the following TM in turn:
- **On input $\langle M, w \rangle$:**
 1. Create $P(M, w)$.
 2. Run PCP on $P(M, w)$.
 - If there's a match, accept.
 - If there's not, reject.
- This TM accepts w if w is accepted by M and rejects it if not; in other words, it decides A_{TM} .
- Therefore, A_{TM} is decidable. $\rightarrow \leftarrow$

Coming Up

- We've introduced the notion of computational histories. This is a powerful tool to map different models of computation to each other, and you'll see it pop up again and again.
- We've also begun, indirectly, to explore the implications of verifying that a problem can be solved. This will show up again.
- Finally, we've begun to explore the idea of problems other than languages. We mapped PCP to a language to deal with it, but it should be pretty clear by now that we didn't really *have* to. Next section, we're going to explore that directly.

Mapping

Wherein We Get To Formally Defining Reducibility After a Surprising Amount of Time Actually Doing It First

Computable Functions

- We started out our exploration of PCP looking for a *solution* to it, not just to recognize the existence of one. We proved that we wouldn't be able to find it, but that's the first step into a world of problems beyond languages.
- Turing machines can do more than recognize languages. In fact, again, they can do anything.
- **Definition: Computable Function**
- A function $f: \Sigma^* \rightarrow \Sigma^*$ is a **computable function** if some Turing machine M , on every input s , halts with only $f(s)$ on its tape.

Notes on Computable Functions

- All arithmetic functions on integers are easily computable.
 - (Hint: If you need to convince yourself, just split two binary numbers with a separator, grab some states, open up the transition function and start building NAND gates.)
- Computable functions can *also* transform Turing machines. We've hinted at this before with the idea of building specific Turing machines to solve specific problems, or to prove we can't.
 - Consider a Turing machine M .
 - Let $f(s) = \langle M' \rangle$ if s is $\langle M \rangle$ for some valid TM M , where M' recognizes the same language as M , but never attempts to move its head off the left-hand end of its tape; and ε if s is not an encoded TM.
 - (Remember this one?)
- All of this gives us a way to finally get to a formal definition of reducibility.

Mapping Reducibility

- To indicate that language A is **mapping reducible** to another language B , we write:

$$A \leq_m B$$

$A \leq_m B$ if there's a computable function $f: \Sigma^* \rightarrow \Sigma^*$ so that for every s :

$$s \in A \iff f(s) \in B.$$

f is called the **reduction** from A to B .

- This formalizes the notion of reducibility we've been working with for the last several sessions:
 - A language is reducible to another language if there is a function that so reduces it.
 - That function is usable for these purposes if it's computable.
 - A function is computable if you can make a Turing machine that computes it.
- You were probably already thinking somewhere along these lines, but now that we've got this definition down formally, we can make some further observations.

Mapping Identities

- If $A \leq_m B$ and B is decidable, A is decidable.
- If $A \leq_m B$ and A is undecidable, B is undecidable.
- If $A \leq_m B$ and B is Turing-recognizable, A is Turing-recognizable.
- If $A \leq_m B$ and A is unrecognizable, B is unrecognizable.
- $A \leq_m B \Leftrightarrow \bar{A} \leq_m \bar{B}$.
 - (Yes, we already knew all this!)
 - (But again, this lets us formalize it neatly.)

Mapping the Halting Problem

- We've already proven the halting problem is unsolvable; here's another way to do so.
 - We want a computable function that transforms an instance of A_{TM} into an instance of $HALT_{TM}$.
 - In other words, we want a Turing machine F that computes $\langle M', s' \rangle$ so that:
$$\langle M, s \rangle \in A_{TM} \iff \langle M', s' \rangle \in HALT_{TM}$$
- The Turing machine F on the right does that just fine. Don't get confused by the order of the mapping – we are turning the input for A into input for $HALT$.
 - Key realization 1: That means we **want** M' to loop on all rejections.
 - Key realization 2: **M' is already going to loop if M does.**

Assume BWOC that $HALT$ is decidable, therefore M_{HALT} exists.

Construct F = On input $\langle M, s \rangle$:

- Construct M' as follows:
 M' = On input x :
 1. Run M on x .
 2. If M accepts, accept.
 3. If M rejects, enter a loop.
- Output $\langle M', s \rangle$.

Then $M_A(\langle M, s \rangle) = M_{HALT}(F(\langle M, s \rangle))$ accepts iff M accepts s , and A_{TM} is decidable. $\Rightarrow \Leftarrow$ and M_{HALT} cannot exist; $HALT$ cannot be decidable.

Mapping Equivalence

EQ_{TM} is unrecognizable:

Reduce $A_{TM} \leq_m \overline{EQ_{TM}}$

F = On input $\langle M, s \rangle$:

- Construct these two machines:
 - M_1 = On any input, reject.
 - M_2 = On any input, run M on s . If it accepts, accept.
- Output $\langle M_1, M_2 \rangle$.

The mapping works like this:

- If M accepts s , M_2 accepts everything, so the machines aren't equivalent.
- If M doesn't accept s , M_2 accepts nothing, so the machines are equivalent.

So M accepts s iff M_1 is not equivalent to M_2 , as desired.

EQ_{TM} is not co-recognizable

Reduce $A_{TM} \leq_m EQ_{TM}$

G = On input $\langle M, s \rangle$:

- Construct these two machines:
 - M_1 = On any input, accept.
 - M_2 = On any input, run M on s . If it accepts, accept.
- Output $\langle M_1, M_2 \rangle$.

The mapping works like this:

- If M accepts s , M_2 accepts everything, so the machines are equivalent.
- If M doesn't accept s , M_2 accepts nothing, so the machines aren't equivalent.

So M accepts s iff M_1 is equivalent to M_2 , as desired.



Acknowledgement

Some Notes and content come from Dr. Gerber, Dr. Hughes, and Mr. Guha's COT4210 class and the Sipser Textbook, *Introduction to the Theory of Computation*, 3rd ed., 2013